

BoxVoyage

Documentation Technique

Projet de fin de L3 Informatique — Box Certificative Finale
Chahine Bensafia — Ilan Sarahoui — Victorian Da Cunha
2025/2026

1. Contexte du projet

BoxVoyage est une application console en Python permettant à des utilisateurs de gérer des lieux d'intérêt géographiques et de construire des circuits de voyage optimisés. Le projet a été développé dans le cadre de la Box Certificative Finale de L3 Informatique.

1.1 Objectifs fonctionnels

- Authentification sécurisée (inscription / connexion)
- Gestion d'une liste de lieux personnels avec géocodage automatique
- Création et gestion de circuits (tours) multi-étapes
- Optimisation de l'ordre de visite des villes (algorithme TSP)
- Gestion d'hôtels et d'excursions journalières depuis un hôtel
- Partage de circuits publics entre utilisateurs

1.2 Interprétations du sujet

Le sujet mentionne « optimisation d'un circuit » sans préciser l'algorithme. Nous avons choisi une approche en deux phases : Nearest Neighbor pour construire rapidement une solution initiale, puis 2-opt pour l'améliorer localement. Cette combinaison offre un bon équilibre entre qualité de la solution et temps de calcul pour des instances de taille courante (< 50 villes).

Le concept d'hôtel n'étant pas explicitement défini, nous l'interprétons comme une ville base depuis laquelle le voyageur effectue des excursions journalières vers des villes proches (rayon configurable, défaut 60 km aller simple). Les villes d'excursion restent dans la liste des places afin de ne pas être perdues si l'hôtel est retiré ultérieurement.

2. Architecture du système

2.1 Vue d'ensemble — architecture en 3 couches

Le projet suit une séparation stricte en trois couches :

Couche	Package	Rôle
Présentation	src/UI/	Interface console, menus, saisies utilisateur
Métier (core)	src/core/	Logique applicative, algorithmes, entités
Données	src/data/	Lecture/écriture JSON, abstraction du stockage

2.2 Structure des fichiers

```
BoxVoyage/
├── main.py
├── src/
│   ├── UI/
│   │   └── console.py
│   ├── core/
│   │   ├── algorithm.py
│   │   ├── auth.py
│   │   ├── places.py
│   │   └── tours.py
│   └── data/
│       └── storage.py
└── dataBase/
    ├── users.json
    ├── places.json
    └── tours.json
```

2.3 Dépendances entre modules

Les couches supérieures dépendent des couches inférieures, jamais l'inverse.

```
console.py
├── auth.py          (AuthManager)
├── places.py        (PlaceManager)
├── tours.py         (TourManager)
└── algorithm.py     (TourOptimischer)

places.py ► algorithm.py (GeoPoint)
tours.py  ► places.py    (Place)
auth.py, places.py, tours.py ► storage.py
```

2.4 Format de stockage JSON

Chaque fichier JSON est un dictionnaire indexé par nom d'utilisateur.

```
# tours.json
{
  "alice": [
```

```
{
  "id": "a1b2c3d4",
  "name": "Tour Europe",
  "is_public": false,
  "places": [{...}],
  "hotels": ["Paris"]
}
]
```

3. Choix technologiques

Technologie	Usage	Justification
Python 3.10+	Langage principal	Syntaxe claire, bibliothèques riches, imposé par le sujet
geopy / Nominatim	Géocodage	API gratuite OpenStreetMap, aucune clé requise
json (stdlib)	Persistance	Format lisible, suffisant pour un usage mono-utilisateur
hashlib (stdlib)	Sécurité mots de passe	SHA-256 natif, pas de dépendance externe
uuid (stdlib)	Identifiants tours	Génère des IDs uniques sans base de données

Le choix d'un stockage JSON plutôt qu'une base de données relationnelle est délibéré : le projet est mono-poste, les volumes de données sont faibles, et JSON offre une inspection visuelle facile.

4. Description des algorithmes

4.1 Distance orthodromique (grande-cercle)

Fichier : `src/core/algorithm.py` — `DistanceCalculator.distance()`

Pour calculer la distance réelle entre deux points GPS, nous utilisons la formule de la grande-cercle (orthodromie) :

```
D = R x arccos( sin(lat1) x sin(lat2) + cos(lat1) x cos(lat2) x cos(lon2 - lon1) )  
Avec R = 6 378.197 km, angles en radians.
```

Les coordonnées sont stockées en degrés décimaux et converties en radians via `lat_rad / lon_rad` de `GeoPoint`. Un clamping dans `[-1, 1]` est appliqué avant `arccos` pour éviter des erreurs numériques sur des points quasi-identiques.

4.2 Nearest Neighbor (heuristique)

Fichier : `TourOptimiser.nearest_neighbor()`

Objectif : construire rapidement un ordre de visite raisonnable pour le problème du voyageur de commerce (TSP).

- On part d'une ville de départ (par défaut la première).
- À chaque étape, on va vers la ville non encore visitée la plus proche.
- On répète jusqu'à avoir visité toutes les villes, puis on retourne au départ.

Complexité : $O(n^2)$. Sert de solution initiale pour 2-opt. Tour retourné fermé : `[A, B, C, A]`.

4.3 Amélioration 2-opt

Fichier : `TourOptimiser.two_opt()`

Objectif : améliorer itérativement le tour en supprimant les croisements de segments. Pour chaque paire d'indices `(i, k)`, on inverse le segment `[i..k]` et on vérifie si la distance totale diminue. Si oui, on conserve et on recommence depuis le début. On s'arrête quand aucun échange n'améliore le résultat.

```
Tour initial : [Paris, Lyon, Marseille, Bordeaux]  
Après inversion [Lyon, Marseille] :  
[Paris, Marseille, Lyon, Bordeaux] <- distance réduite
```

Complexité : $O(n^2)$ par itération, $O(n^3)$ dans le pire cas. Converge vers un optimum local.

4.4 Pipeline complet d'optimisation

Fichier : `TourOptimiser.optimize_places_with_distances()`

- Conversion des dicts de places en objets `GeoPoint`
- Nearest Neighbor -> tour fermé `[A...Z, A]`
- Suppression du doublon final `[:-1]` avant 2-opt (tour ouvert requis)
- 2-opt -> tour amélioré

- Reconstruction des dicts de places depuis les GeoPoints optimisés
- Calcul des distances par segment et distance totale aller-retour

Retourne un dict : `optimized_places`, `segments`, `total_distance`.

4.5 Gestion des hôtels et excursions

Fichier : `TourOptimiser.split_by_hotels()`

Étape 1 — Partitionnement :

- Pour chaque ville non-hôtel, on calcule sa distance à chaque hôtel.
- Si la distance au plus proche hôtel est $\leq$ rayon (défaut 60 km), la ville devient une excursion de cet hôtel.
- Sinon, elle reste dans le tour principal. Les hôtels eux-mêmes sont toujours dans le tour principal.

Étape 2 — Optimisation du tour principal uniquement :

- On appelle `optimize_places_with_distances` sur `main_tour` seulement.
- Les villes d'excursion ne participent pas à la boucle principale.

Les villes d'excursion sont conservées dans `places` du tour. Si l'hôtel est retiré et que l'on recalcule, elles réintègrent automatiquement le tour principal.

5. Sécurité

5.1 Hachage des mots de passe

Les mots de passe ne sont jamais stockés en clair. À l'inscription, le mot de passe est haché avec SHA-256 (hashlib) et seul le hash hexadécimal est persisté dans users.json.

```
password_hash = hashlib.sha256(password.encode()).hexdigest()  
# Résultat : chaîne hex de 64 caractères
```

À la connexion, le même hachage est appliqué au mot de passe saisi et comparé au hash stocké.

5.2 Bypass de __init__ à la désérialisation

Lors du chargement depuis JSON, les classes Users et Tour utilisent cls.__new__(cls) pour contourner __init__. Cela évite de re-hacher un hash déjà haché (Users) ou de générer un nouvel UUID (Tour).

6. Gestion du code source

6.1 Branches Git

Le développement a été organisé en trois branches :

- feat/vic (Victorian Da Cunha) : authentification, stockage, console de base, algorithmes TSP
- feat-ilan (Ilan Sarahoui) : API géocodage, algorithmes TSP, ville de départ
- Chahine (Chahine Bensafia) : places, tours, hôtels/excursions, intégration console, documentation

Un conflit de fusion sur console.py a été résolu manuellement : feat-ilan avait ajouté le choix de la ville de départ (option 5), la branche Chahine avait ajouté la gestion des hôtels. Le menu final intègre les deux : 5=ville de départ, 6=visibilité, 7=hôtels, 8=retour.

6.2 Convention d'imports

Tous les imports utilisent le préfixe src. (from src.core.algorithm import ...) car le projet est exécuté depuis la racine avec python main.py. Sous Linux, src/core/ et src/Core/ sont des répertoires distincts (sensibilité à la casse) : le projet utilise src/core/ (minuscule) de façon cohérente.

7. Instructions d'installation et de mise en route

7.1 Prérequis

- Python 3.10 ou supérieur
- pip (inclus avec Python)
- Accès Internet (géocodage via Nominatim)

7.2 Installation

```
git clone <url_du_depot>
cd BoxVoyage
pip install geopy
python main.py
```

7.3 Arborescence de données

Au premier lancement, le dossier dataBase/ est créé automatiquement avec des fichiers JSON vides. Aucune configuration manuelle n'est requise.

7.4 Utilisation rapide

1. S'inscrire ou se connecter.
2. Ajouter des lieux (menu principal -> option 1).
3. Créer un tour depuis sa liste de lieux (option 2).
4. Dans le tour : ajouter des villes, désigner des hôtels, optimiser.
5. Consulter le résumé avec distances par segment.

8. Répartition du travail en équipe

Le projet a été développé en équipe de trois étudiants, chacun sur une branche dédiée avant intégration sur la branche principale.

Membre	Branche	Contributions principales
Victorian Da Cunha	feat/vic	Authentification (auth.py), stockage (storage.py, dataBase/), base de la console (console.py), algorithmes TSP (nearest neighbor, 2-opt)
Ilan Sarahoui	feat-ilan	Intégration API géocodage (Nominatim), algorithmes TSP (participation), ville de départ dans l'optimisation
Chahine Bensafia	Chahine	Gestion des lieux (places.py), gestion des tours (tours.py), système hôtels/excursions (split_by_hotels), intégration console hôtels, documentation technique

Les algorithmes d'optimisation ont été compris et travaillés collectivement, même si Victorian Da Cunha et Ilan Sarahoui en ont principalement écrit le code. La logique hôtels/excursions a été développée et intégrée par Chahine Bensafia.

8.1 Gestion des branches et fusion

Chaque membre a développé sur sa branche dédiée. La fusion vers main a nécessité la résolution d'un conflit sur console.py : la branche feat-ilan avait ajouté le choix de la ville de départ (menu option 5) tandis que la branche Chahine avait ajouté la gestion des hôtels. Le menu final intègre les deux fonctionnalités.

9. Difficultés rencontrées et limites

9.1 Difficultés rencontrées

Imports et convention de chemins (src.core)

Un problème récurrent en début de projet a été la gestion des imports Python. Le projet étant lancé depuis la racine avec `python main.py`, tous les imports devaient utiliser le préfixe `src`. (`from src.core.algorithm import ...`). Des erreurs `ModuleNotFoundError` sont apparues lorsque certains fichiers utilisaient `from core.xxx` au lieu de `from src.core.xxx`, ou importaient depuis `src.Core` (majuscule) — répertoire distinct sous Linux où le système de fichiers est sensible à la casse. Ces erreurs ont été corrigées en uniformisant tous les imports.

Fusions de branches (merge conflicts)

Plusieurs conflits de fusion ont été rencontrés lors de l'intégration des branches. Le plus significatif portait sur `console.py` : la branche d'Ilan avait ajouté la fonctionnalité de ville de départ (option 5 du menu) tandis que la branche de Chahine avait ajouté la gestion des hôtels. Les deux fonctionnalités modifiaient le même bloc de code. Le conflit a été résolu manuellement. Des conflits sur `__pycache__` ont également été résolus en ajoutant les exclusions dans le `.gitignore`.

Compréhension et mise en place des algorithmes

L'implémentation des algorithmes d'optimisation a représenté la partie la plus exigeante du projet. Il a fallu comprendre le problème du voyageur de commerce (TSP), choisir une approche adaptée (Nearest Neighbor + 2-opt), puis implémenter correctement la formule de distance orthodromique avec gestion des erreurs numériques.

La difficulté s'est accrue avec l'ajout de la contrainte hôtels en cours de projet. Il a fallu adapter l'architecture existante sans casser les fonctionnalités déjà en place : introduire `split_by_hotels`, modifier le pipeline pour n'optimiser que le tour principal, conserver les villes d'excursion dans places, et gérer l'affichage inline. La question du rayon (aller simple vs aller-retour) a également nécessité une réflexion sur la sémantique attendue.

9.2 Fonctionnalité non implémentée : verrouillage de position

Par manque de temps, nous n'avons pas pu implémenter la possibilité de fixer une ville à un index précis dans le tour. Techniquement, cela aurait nécessité un champ `locked: true` dans le dict de place et une vérification dans la boucle `two_opt` pour ne pas inverser un segment contenant une ville verrouillée.

9.3 Limites techniques

- Stockage JSON non concurrent : en usage mono-utilisateur ce n'est pas un problème, mais deux instances simultanées pourraient corrompre les données.
- SHA-256 sans sel (salt) : suffisant pour un projet académique, mais en production `bcrypt` ou `argon2` avec sel seraient indispensables.
- 2-opt : optimum local uniquement. Pour de très grands circuits, Lin-Kernighan ou des algorithmes génétiques donneraient de meilleurs résultats.

9.4 Pistes d'amélioration

- Implémenter le verrouillage de position de ville dans le tour.
- Ajouter une interface web (Flask/FastAPI) en conservant la couche core intacte.
- Intégrer une carte interactive (Folium) pour visualiser les tours.
- Ajouter des tests unitaires (pytest) sur les algorithmes et le stockage.